



机器视觉实验课程设计二

基于 ConvNeXt 迁移学习的六类手势识别实验报告

姓 名：冯怡然

学 号：2411434

班 级：人工智能特色班

日 期：2026 年 5 月 23 日

南开大学 · 机器视觉实验

1 实验目的

本实验面向六类手势识别任务，类别包括 A、B、C、Five、Point 和 V。作业要求使用深度学习方法对简单背景和复杂背景下的手势图片进行分类，评价标准以识别准确率为主。因此，本实验的目标不是单纯搭建一个能够运行的分类器，而是在有限训练数据和仅使用 CPU 训练的条件下，尽可能提升模型在不同拍摄风格图片上的泛化能力。

经过对 MobileNetV3、EfficientNet-B0 与 ConvNeXt-Tiny 的对比实验，最终选择 ConvNeXt-Tiny 作为提交模型。选择该模型的原因是：它保留了卷积神经网络对局部视觉特征的建模优势，同时吸收了近年视觉 Transformer 中较有效的宏观与微观的设计思想，在本实验的自建测试集上取得了更高准确率。

2 数据集与任务设置

实验数据按照 PyTorch 常用的 ImageFolder 结构组织，每个类别对应一个文件夹。训练集与验证集来自课程提供的数据，另设置一个 TestImages 文件夹用于模拟实际提交前的额外测试。TestImages 里面都是本人拍摄的不同同学的手势图片。数据类别分布如表 1 和图 1 所示。

表 1: 训练集、验证集与 TestImages 数量统计

数据集	A	B	C	Five	Point	V	合计
训练集	1062	406	483	566	1115	400	4032
验证集	265	102	121	141	279	100	1008
TestImages	8	5	7	9	6	11	46

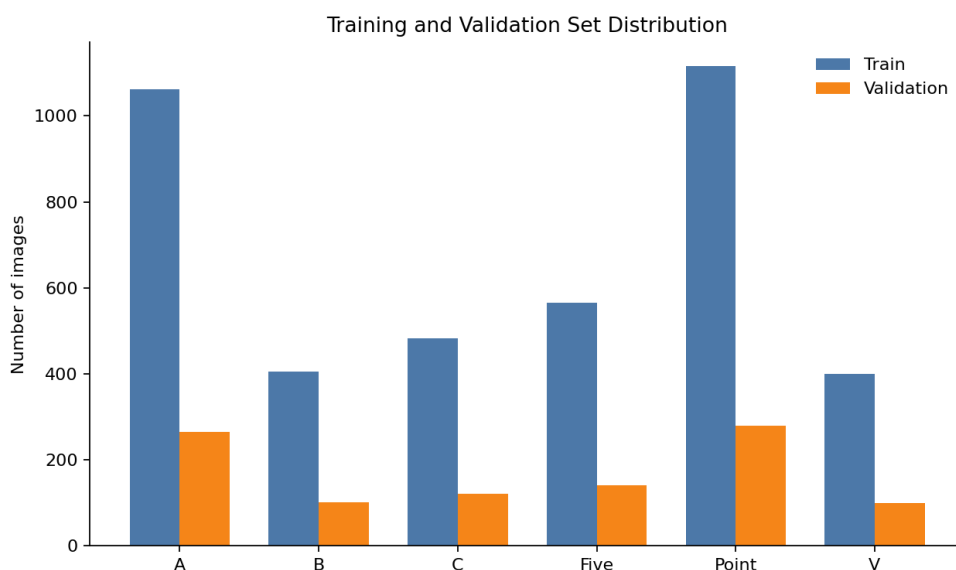


图 1: 训练集与验证集各类别图片数量分布

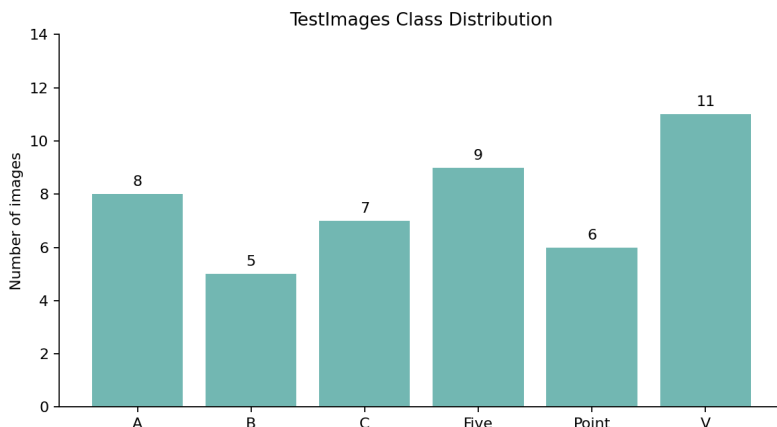


图 2: TestImages 各类别图片数量分布

从数据分布可以看出，Point 和 A 类图片较多，B 和 V 类图片相对较少。类别数量不完全均衡会使模型更容易偏向样本多的类别，因此训练时加入了类别权重，以减轻类别不平衡对损失函数的影响。

3 实验原理

3.1 ConvNeXt 模型思想

本实验最终采用的模型为 ConvNeXt-Tiny。ConvNeXt 来自 Liu 等人在论文 *A ConvNet for the 2020s* 中提出的卷积网络改进思路。该工作的核心思想是：传统卷积神经网络并没有过时，只要结合现代视觉模型的训练方法和结构设计，卷积网络仍然可以获得很强的视觉识别能力。

ConvNeXt 以 ResNet 风格的分阶段卷积网络为基础，但对网络结构进行了系统化改造。与普通 ResNet 相比，ConvNeXt 更强调大核深度卷积、Inverted Bottleneck、LayerNorm 和更简洁的激活函数使用方式。这些改动使模型既能保持卷积网络对图像局部纹理、边缘和形状的敏感性，又能在较深层网络中扩大感受野，从而更好地识别复杂背景下的手势区域。

3.2 ConvNeXt Block

ConvNeXt 的一个基本模块可以概括为：

1. 使用深度可分离卷积提取空间特征。论文中常用 7×7 大卷积核，使单个卷积层能够看到更大范围的图像上下文。
2. 使用 LayerNorm 进行归一化。相比传统 CNN 中常见的 BatchNorm，LayerNorm 对小 batch 场景更友好，本实验在 CPU 上训练，batch size 较小，这一点比较适合。
3. 使用两个 1×1 点卷积完成通道维度的扩张与压缩，形成 Inverted Bottleneck。
4. 使用 GELU 作为非线性激活函数，并通过残差连接保留输入特征，减轻深层网络训练难度。

在手势识别任务中，手指数量、手掌轮廓、指尖方向以及手部与背景的边界都是重要判别信息。ConvNeXt 的大核深度卷积有助于同时观察手部局部细节和整体姿态，因此比轻量化模型更适合本实验中背景变化较大的图像。

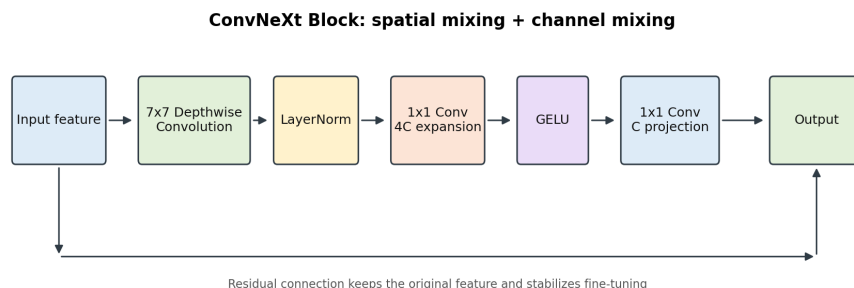


图 3: ConvNeXt Block 结构示意图

3.3 迁移学习策略

由于课程数据集规模有限，如果完全从零开始训练 ConvNeXt，模型容易过拟合，并且在 CPU 上训练时间较长 (本人没有 GPU 的电脑)。因此本实验采用 ImageNet 预训练权重进行迁移学习。具体做法为：

1. 加载 torchvision 提供的 ConvNeXt-Tiny ImageNet 预训练模型。
2. 将原始分类头替换为输出 6 个类别的线性层。
3. 前 5 个 epoch 冻结主干网络，只训练新的分类头，使模型先适应手势类别。
4. 后续 epoch 解冻全部参数，用较小学习率进行微调，使高层语义特征进一步贴合手势识别任务。

这种训练方式相当于先利用大规模自然图像中学到的通用边缘、纹理和形状特征，再针对本实验的手势数据进行定制化调整。

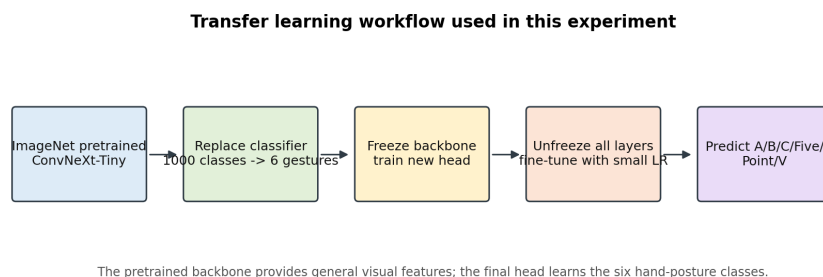


图 4: 本实验采用的 ConvNeXt 迁移学习流程示意图

4 实验方法

4.1 预处理与数据增强

训练阶段输入图像统一处理为 224×224 。为了提高模型对拍摄角度、背景和光照变化的鲁棒性，训练时采用如下数据增强方式：

- 随机裁剪并缩放图像，模拟手部在画面中大小和位置的变化；
- 随机水平翻转，增强左右方向变化；
- 随机旋转，适应手势倾斜；
- 亮度、对比度、饱和度和色调扰动，模拟不同拍摄环境；
- Random Erasing，模拟局部遮挡；
- 使用 ImageNet 均值和标准差进行归一化。

验证和测试阶段不使用随机增强，只采用确定性的 resize、center crop 和归一化，保证评估结果可复现。

4.2 训练配置

最终 ConvNeXt-Tiny 模型的主要训练配置如表 2 所示。

表 2: ConvNeXt-Tiny 训练配置

项目	设置
框架	PyTorch + torchvision
设备	CPU
输入尺寸	224×224
预训练权重	ImageNet1K
epoch 数	25
冻结训练	前 5 个 epoch 只训练分类头
batch size	4
优化器	AdamW
初始学习率	3×10^{-4}
微调学习率	8×10^{-5}
权重衰减	0.05
Label smoothing	0.05
类别权重	启用

训练命令如下：

```
python train.py --model convnext_tiny --epochs 25 --freeze-epochs 5 ^
--batch-size 4 --workers 0 --class-weight ^
--out-dir runs\convnext_tiny_cpu
```

4.3 测试方式

测试阶段使用训练得到的最佳权重 `runs\convnext_tiny_cpu\best.pth`。对 TestImages 测试时，采用 crop 模式和 TTA 测试时增强。TTA 会对同一张图片进行原图和水平翻转预测，再融合结果，可在不重新训练的情况下略微提升稳定性。

```
python predict_folder.py TestImages ^
--checkpoint runs\convnext_tiny_cpu\best.pth ^
--resize-mode crop --tta --out predictions_convnext.csv
```

5 实验结果

5.1 训练过程

ConvNeXt-Tiny 的训练曲线如图 5 所示。前 5 个 epoch 冻结主干网络时，模型主要学习新的六分类头；第 6 个 epoch 之后解冻主干网络，验证集准确率明显上升，说明迁移学习和微调策略有效。

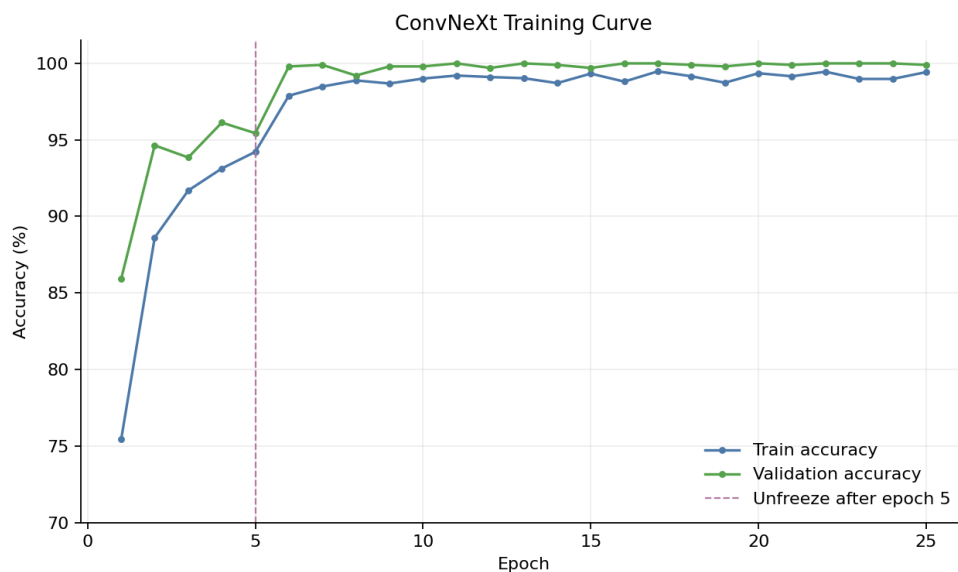


图 5: ConvNeXt-Tiny 训练与验证准确率曲线

最终训练结束时，ConvNeXt-Tiny 在验证集上的准确率为 99.90%，训练过程中最好验证准确率达到 100.00%。验证集与训练集来自同一数据来源，因此该结果主要说明模型已经充分学习了课程数据中的手势特征。

5.2 不同模型对比

为了说明最终选择 ConvNeXt 的原因，本实验比较了 MobileNetV3-Large、EfficientNet-B0 和 ConvNeXt-Tiny 三种模型。三种模型在验证集上均能达到接近满分的结果，但在 TestImages 上差异更明显。对比结果如表 3 和图 6 所示。

表 3: 不同模型在验证集与 TestImages 上的结果对比

模型	验证集最好准确率	TestImages 准确率	TestImages 正确数
MobileNetV3-Large	100.00%	69.57%	32/46
EfficientNet-B0	100.00%	80.43%	37/46
ConvNeXt-Tiny	100.00%	91.30%	42/46

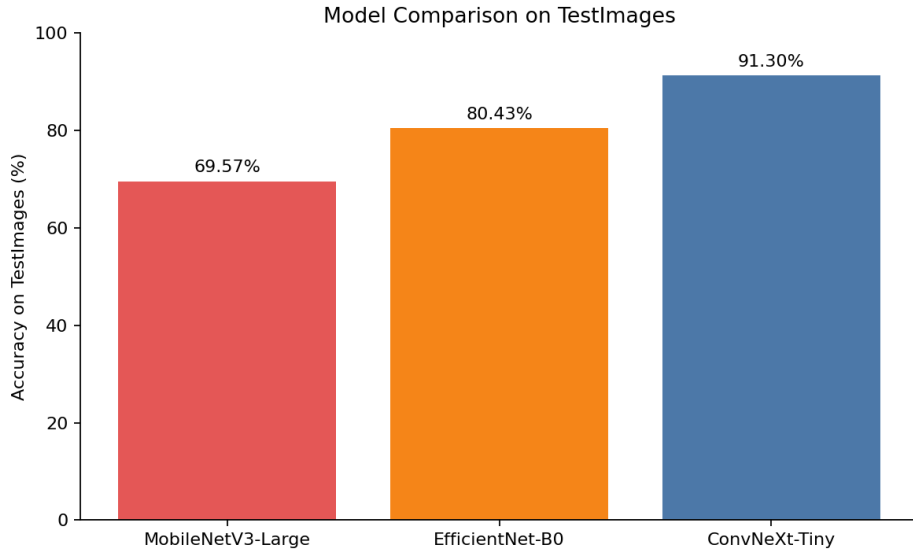


图 6: 三种模型在 TestImages 上的准确率对比

从结果看, ConvNeXt-Tiny 在 TestImages 上正确识别 42 张, 共 46 张, 准确率为 91.30%, 相比 EfficientNet-B0 提高约 10.87 个百分点, 相比 MobileNetV3-Large 提高约 21.73 个百分点。因此, 最终提交模型选择 ConvNeXt-Tiny。

5.3 ConvNeXt 在 TestImages 上的分类表现

ConvNeXt-Tiny 在 TestImages 上的分类报告如表 4 所示。

表 4: ConvNeXt-Tiny 在 TestImages 上的分类指标

类别	Precision	Recall	F1-score	Support
A	1.0000	0.8750	0.9333	8
B	0.8333	1.0000	0.9091	5
C	0.7778	1.0000	0.8750	7
Five	1.0000	1.0000	1.0000	9
Point	0.8571	1.0000	0.9231	6
V	1.0000	0.7273	0.8421	11
Accuracy	0.9130			
Macro avg	0.9114	0.9337	0.9138	46
Weighted avg	0.9294	0.9130	0.9117	46

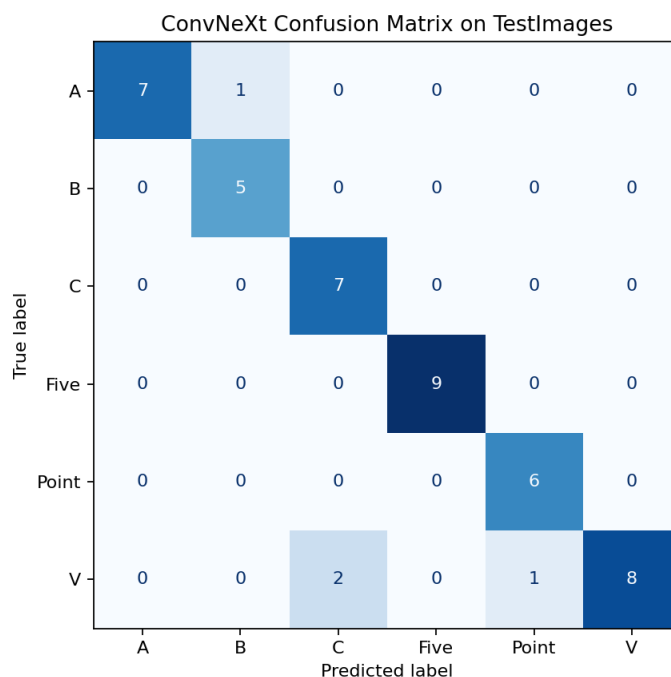


图 7: ConvNeXt-Tiny 在 TestImages 上的混淆矩阵

从混淆矩阵可以看到，Five 类识别最稳定，全部识别正确。A、B、C、Point 类整体表现较好，主要错误集中在 V 类。V 类的召回率为 72.73%，说明部分真实 V 手势在特定角度或背景下容易被识别为 C 或 Point。原因可能是 V 手势与 Point 都具有明显的指尖方向特征，而某些图片中两根手指分离不够明显时，模型会更依赖局部轮廓，从而产生混淆。

6 结果分析

本实验中，三种模型在验证集上都取得了很高准确率，但 TestImages 上的差距较大。这说明验证集和训练集风格较接近，能够反映模型是否学会了课程数据中的主要模式；而 TestImages 更能反映模型面对新拍摄图像时的泛化能力。

MobileNetV3-Large 的优势是速度快、参数量小，但在本任务中对复杂背景和细小手指形态的区分能力不足。EfficientNet-B0 比 MobileNetV3-Large 更稳定，但对 A、V 等类别的召回率仍然偏低。ConvNeXt-Tiny 的准确率最高，说明其更强的特征表达能力能更好地处理背景变化、手部位置变化和手势形态细节。

不过，ConvNeXt-Tiny 仍然存在可改进空间。首先，V 类仍是主要错误来源，后续可以增加更多 V 类不同角度、不同背景、不同距离的图片。其次，当前 TestImages 数量为 46 张，规模仍然较小，后续可以继续扩充测试图片以获得更稳定的评估。最后，如果有 GPU 条件，可以尝试更长时间微调、调整学习率，或尝试 ConvNeXt-Small 等更大模型。

7 实验总结

本实验完成了从数据整理、模型训练、迁移学习、模型对比到文件夹预测的完整手势识别流程。最终采用 ConvNeXt-Tiny 作为提交模型，原因是它在验证集上达到 100.00% 最好准确率，并在 TestImages 上达到 91.30% 准确率，优于 MobileNetV3-Large 和 EfficientNet-B0。

从实验过程看，ConvNeXt 适合本任务的主要原因可能在于它结合了卷积网络的局部特征提取能力和现代视觉模型的大感受野设计。对于手势识别而言，模型既需要识别手指数量和指尖方向等局部细节，也需要理解整只手的姿态轮廓，ConvNeXt 在这两方面取得了较好的平衡。

这次做大作业训练了很多不同的模型，也阅读了不少论文。从读论文的角度上，我深刻的感受到从训练、宏观结构和微观结构各个视角借鉴 Transformer 的思想来调参数这件事情对于模型表现的重要影响，并且发现 Transformer 和 CNN 居然在很多地方如此相通。

此外，在从 MobileNet 到 EfficientNet 再到 ConvNeXt 的测试中，我深刻的体会到近年来机器视觉模型的迅速发展。同时在不断地调试当中，我也熟悉了训练模型的基本过程，看到模型训练迭代和一次次的调整，也让我反思自己的学习过程，对学习这些事本身有了更深的体会。

参考文献

- [1] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. *A ConvNet for the 2020s*. arXiv:2201.03545v2, 2022.
- [2] TorchVision documentation. *ConvNeXt model builders and ConvNeXt-Tiny pretrained weights*. <https://docs.pytorch.org/vision/main/models/convnext.html>